

Chapter 3: CPU

Exercise 1: Problem Scenario

Explain sample problem with answer on Program Execution in the fetch execute cycle of computer system: memory 200 with a value of 100, memory 201 with a value of 150, memory 202 with a value of 175, memory 500 with a value of 10, 501 with a value of 5. Program: 0005 = Load AC from Memory; 0010 = Store AC to Memory; 0101 = Add to AC from Memory.

SOLUTION: Fetch-Execute Cycle OR Instruction Cycle)

A typical computer architecture question provides the state of the memory and the CPU's instruction set. Formalize the given:

1. Data Memory Locations:

- **Memory 500** = 10 (Variable A)
- **Memory 501** = 5 (Variable B)

2. Instruction Set Architecture (Opcodes):

- **0005 = Load AC from Memory** (Loads a value from a specified memory address into the Accumulator register).
- **0101 = Add to AC from Memory** (Adds a value from a specified memory address to the current value in the Accumulator).
- **0010 = Store AC to Memory** (Copies the value in the Accumulator to a specified memory address).

3. Program Instructions (Stored sequentially starting at address 200):

An instruction format in this scenario combines the **Opcode** and the **Target Address**.

- **Memory 200:** 0005 500 (*Load the value from address 500 into AC*)
- **Memory 201:** 0101 501 (*Add the value from address 501 to AC*)
- **Memory 202:** 0010 500 (*Store the resulting AC value back into address 500*)

Before starting, the **Program Counter (PC)** is initialized to **200** (the starting address of our program), and the **Accumulator (AC)** is empty (0).

Cycle 1: Processing Instruction at Memory 200

- **Fetch:** The CPU checks the PC (200) and retrieves the instruction 0005 500 from memory address 200 into the Instruction Register (IR). The **PC automatically increments to 201**.
- **Decode:** The Control Unit decodes 0005 as a **Load AC** operation and identifies 500 as the target data address.
- **Execute:** The CPU reads the value at memory address 500 (which is 10) and loads it into the Accumulator.
 - *End of Cycle 1 Status:* PC = 201, AC = 10, Memory 500 = 10

Cycle 2: Processing Instruction at Memory 201

- **Fetch:** The CPU reads the PC (201) and fetches the instruction 0101 501. The PC increments to 202.
- **Decode:** The Control Unit decodes 0101 as an Add to AC operation targeting memory address 501.
- **Execute:** The CPU fetches the value at address 501 (which is 5). The Arithmetic Logic Unit (ALU) adds this to the current AC value (10 + 5). The result is kept in the Accumulator.
 - *End of Cycle 2 Status:* PC = 202, AC = 15, Memory 500 = 10

Cycle 3: Processing Instruction at Memory 202

- **Fetch:** The CPU reads the PC (202) and fetches the instruction 0010 500. The PC increments to 203.
- **Decode:** The Control Unit decodes 0010 as a Store AC operation targeting memory address 500.
- **Execute:** The CPU takes the current value inside the AC (15) and writes it directly into memory address 500, overwriting the old value.
 - *End of Cycle 3 Status:* PC = 203, AC = 15, Memory 500 = 15

Final Answer Summary

After completing all three fetch-execute cycles, the final states of the registers and modified memory locations are:

Component	Initial State	Final State
Program Counter (PC)	200	203
Accumulator (AC)	0	15
Memory Location 500	10	15
Memory Location 501	5	5 (Unchanged)

Exercise 2: Problem Scenario

Consider a 32-bit, byte-addressable computer system.

- **Instruction Word Size:** 32 bits (4 bytes).
- **Current Program Counter (PC):** 0x0000 1000
- **Status Register:** Zero Flag (Z) = 1 (indicating the last arithmetic operation resulted in zero).

Instruction Format:

The CPU uses a 32-bit fixed-length instruction format for branch operations:

- **Opcode (8 bits):** Specifies the operation (e.g., 0x74 for JZ - Jump if Zero).
- **Condition Bits (4 bits):** Specifies which flags to check.
- **Immediate Offset (20 bits):** A signed two's complement integer representing the target address offset *relative to the next instruction* (PC + 4).

Memory Contents:

At memory address 0x0000 1000, the following 32-bit instruction is stored:
0x7410 0008

SOLUTION:

Step through the **Instruction Cycle** (Fetch, Decode, Execute) for this instruction. Show the values of the **Program Counter (PC)**, **Instruction Register (IR)**, and the **Next PC** at the end of the cycle.

1. Fetch Phase

- **Action:** The CPU sends the address in the PC (0x0000 1000) to the memory unit via the Address Bus. The memory returns the 32-bit data word stored at that location.
- **Instruction Register (IR) Update:** The fetched instruction is loaded into the IR.
$$IR = 0x7410\ 0008$$
- **PC Increment:** Because this is a 32-bit architecture and memory is byte-addressable, each instruction occupies 4 bytes. The PC automatically increments by 4 to point to the next sequential instruction.

$$PC = 0x0000\ 1000 + 4 = 0x0000\ 1004$$

2. Decode Phase

- **Action:** The Control Unit splits the 32-bit IR into its designated functional fields based on the 32-bit architecture format:
 - **Opcode:** 0x74 (Decoded as a conditional "Jump if Zero" instruction).
 - **Condition:** 0x1 (Instructs the hardware to evaluate the Zero Flag (Z)).
 - **Immediate Offset:** 0x00008 (Sign-extended 20-bit value to a 32-bit value: 0x0000 0008).

3. Execute Phase

- **Condition Evaluation:** The Control Unit checks the Status Register's Zero Flag.
 - Since $Z = 1$, the condition evaluates to **True**. The branch is taken.
- **Target Address Calculation:** The CPU calculates the target branch address by adding the sign-extended offset to the updated PC ($PC_{current} = 0x0000\ 1004$).
$$\text{Target Address} = PC + \text{Offset}$$
$$\text{Target Address} = 0x0000\ 1004 + 0x0000\ 0008 = 0x0000\ 100C$$
- **PC Update:** The calculated target address is loaded directly into the PC, overwriting the sequential address.

Final Register States

At the end of this instruction cycle, the key CPU registers contain:

- **IR:** 0x7410 0008 (The executed branch instruction)
- **PC:** 0x0000 100C (The address of the next instruction to fetch, successfully modified by the conditional branch).

Exercise 3: Problem Statement

A 32-bit computer architecture uses a single-bus CPU organization. An instruction is fetched from memory at address 0x00001000. The instruction is a conditional branch instruction: **BEQ R1, R2, Offset** (Branch if Equal) – [The instruction *beq r1, r2, offset* means **branch if equal**, which compares the values in two registers and jumps to a new location in the program if they match]

Assume the following initial conditions:

- Each instruction word is **32 bits (4 bytes)**.
- **PC** (Program Counter) = 0x00001000
- **R1** = 0x00000005
- **R2** = 0x00000005 (The condition is met, so the branch is taken).
- **Offset** = 0x00000018 (The sign-extended target offset).

Task: Show the complete step-by-step **Instruction Cycle** (Fetch, Decode, Execute) by listing the Micro-operations / Control Sequence, and provide a **Block Diagram** illustrating the final state of program execution.

SOLUTION

Step-by-Step Control Sequence

Because the architecture uses a 32-bit word size, the Program Counter (PC) must automatically increment by 4 bytes during the fetch phase to point to the next sequential instruction.

1. Fetch Phase

- Step 1: $MAR \leftarrow [PC]$ (Move the address from PC to the Memory Address Register).
- Step 2: Read memory, $Z \leftarrow [PC] + 4$ (Send PC to the ALU to calculate the next sequential address).
- Step 3: $PC \leftarrow [Z]$, $MDR \leftarrow \text{Memory}$ (Update PC with the sequential address 0x00001004 while waiting for memory data).
- Step 4: $IR \leftarrow [MDR]$ (Load the 32-bit instruction word into the Instruction Register).

2. Decode Phase

- Step 5: The Control Unit decodes the opcode in the IR. It identifies the instruction as BEQ and extracts the register identifiers (R1, R2) and the branch Offset.

3. Execute Phase (Conditional Branch Taken)

- Step 6: $ALU_A \leftarrow [R1]$ (Load the contents of R1 into the ALU's first input register).
- Step 7: $ALU_B \leftarrow [R2]$, Subtract (Compare R1 and R2 by subtracting them).
- Step 8: Check the ALU Zero Flag (Z-Flag).
 - Since $5 - 5 = 0$, the Z-Flag is set to 1 (True).
 - The Control Unit branches to the target address update sequence.
- Step 9: $ALU_A \leftarrow [PC]$ (Load the current updated PC 0x00001004 into the ALU).
- Step 10: $ALU_B \leftarrow [\text{Offset}]$, Add (Add the sign-extended offset 0x00000018 to the PC).
- Step 11: $PC \leftarrow ALU_Result$ (Update the PC with the final branch target address: 0x0000101C).

Program Execution Block Diagram

The diagram below shows the internal state of the 32-bit CPU architecture at the **end** of the execution cycle.

